E4 Educator v2.0

T11

NVS persistence

Tier 2 · Tutorial 4 of 9

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- What NVS is, how it differs from SD card storage, and when to use each
- How the ESP32 NVS partition works and its capacity limits
- How to use the Preferences library for all basic data types
- The 15-character key rule and why violating it silently fails
- How to organise NVS data using namespaces
- How to build a settings manager that persists across reboots
- How to implement a factory reset that clears all NVS data
- How to combine NVS settings with a TFT settings UI
- NVS versus SD — a clear decision framework for every project

You will need:

- E4 Educator v2.0 board with ESP32 fitted
- T08 completed — TFT and sprite pipeline
- Button.h and TouchZone.h from previous tutorials

1 What is NVS?

NVS stands for Non-Volatile Storage. It is a key-value store built into the ESP32's flash memory that survives power cycles and reboots. Unlike RAM which loses its contents on reset, NVS data persists indefinitely until you explicitly delete it or erase the flash.

You have already used NVS twice in this course without focusing on it directly — in T09 to store touch calibration data, and in T10's config loader as an alternative to SD files. T11 covers it thoroughly so you can apply it confidently in every project.

1.1 NVS versus SD card — when to use each

Property	NVS	SD card
Location	ESP32 internal flash	External microSD card
Always available	Yes — no card required	Only if card inserted and mounted
Capacity	~20KB typical (small)	2GB–32GB (enormous)
Best for	Settings, calibration, counters, WiFi credentials, preferences	Data logging, config files, images, audio, large datasets
Write endurance	~100,000 writes per key	~100,000 erase cycles per block
Access speed	Fast — internal flash	Slower — SPI bus + FAT overhead
Power loss safety	Atomic writes — safe	Must close files — risk if power lost mid-write

Key point

The decision rule is simple: small settings that must survive reboots and work without an SD card go in NVS. Large or growing data goes on SD. Calibration data, brightness preference, WiFi credentials, and alert thresholds belong in NVS. Sensor logs, BMP images, and config files belong on SD.

1.2 The NVS partition

The ESP32's flash memory is divided into partitions. The default partition table allocates approximately 20KB to NVS. This sounds small, but for settings storage it is ample — a typical settings set of 10 values uses less than 1KB.

NVS stores data as key-value pairs in pages of 4KB. Each page can hold around 125 entries. When a page fills, NVS automatically uses the next page. When all pages are full, NVS performs garbage collection — reclaiming space from deleted entries. This is handled transparently.

2 The Preferences library

The Preferences library is the Arduino-friendly wrapper around the ESP-IDF NVS API. It is included with the ESP32 Arduino core — no lib_deps entry needed. The interface is clean and consistent for all data types.

2.1 Basic read and write pattern

```
#include <Arduino.h>
#include <Preferences.h>

Preferences prefs;

void setup() {
  Serial.begin(115200);

  // ── Writing values ──────────────────────────────
  prefs.begin("settings", false);  // namespace, read-write

  prefs.putInt("brightness", 200);
  prefs.putFloat("tempAlert", 28.0);
  prefs.putBool("logEnabled", true);
  prefs.putString("devName", "E4-Logger-01");
  prefs.putULong("bootCount",
                 prefs.getULong("bootCount", 0) + 1);

  prefs.end();  // Always end when done writing

  // ── Reading values ──────────────────────────────
  prefs.begin("settings", true);  // namespace, read-only

  int   brightness = prefs.getInt("brightness", 200);
  float tempAlert  = prefs.getFloat("tempAlert", 28.0);
  bool  logEnabled = prefs.getBool("logEnabled", true);
  String devName   = prefs.getString("devName", "E4-Device");
  unsigned long boots = prefs.getULong("bootCount", 0);

  prefs.end();

  Serial.printf("Brightness: %d\n",   brightness);
  Serial.printf("Temp alert: %.1f\n",  tempAlert);
  Serial.printf("Logging:    %s\n",    logEnabled ? "yes" : "no");
  Serial.printf("Device:     %s\n",    devName.c_str());
  Serial.printf("Boot count: %lu\n",   boots);
}

void loop() {}
```

The second argument to every get() call is the default value returned when the key does not exist yet. This makes first-boot behaviour predictable — the device uses sensible defaults until the user changes a setting.

2.2 Supported data types

Type	put method	get method	Bytes stored
bool	`putBool(key, val)`	`getBool(key, def)`	1
int8_t	`putChar(key, val)`	`getChar(key, def)`	1
uint8_t	`putUChar(key, val)`	`getUChar(key, def)`	1
int16_t	`putShort(key, val)`	`getShort(key, def)`	2
int32_t / int	`putInt(key, val)`	`getInt(key, def)`	4
uint32_t	`putUInt(key, val)`	`getUInt(key, def)`	4
int64_t / long	`putLong64(key, val)`	`getLong64(key, def)`	8
uint64_t / ulong	`putULong(key, val)`	`getULong(key, def)`	8
float	`putFloat(key, val)`	`getFloat(key, def)`	4
double	`putDouble(key, val)`	`getDouble(key, def)`	8
String	`putString(key, val)`	`getString(key, def)`	variable
byte array	`putBytes(key, buf, len)`	`getBytes(key, buf, len)`	variable

3 The 15-character key rule

This is the single most important rule in NVS programming on the ESP32. It is not obvious, the error is silent, and it catches experienced developers as well as beginners.

⚠ NVS keys must be 15 characters or fewer

The ESP32 NVS implementation silently truncates key names to 15 characters. If you use a key longer than 15 characters, the put() appears to succeed but the stored key is truncated. If two long keys share the same first 15 characters, they collide silently — one overwrites the other. No error is raised. The only symptom is wrong values being returned on get(). Always count your key characters before use.

```
// ✗ BAD — all three truncate to "temperatureAler"
prefs.putFloat("temperatureAlert",   28.0);
prefs.putFloat("temperatureAlerts",  28.0);
prefs.putFloat("temperatureAlerting",28.0);

// ✓ GOOD — all within 15 characters
prefs.putFloat("tempAlert",  28.0);  // 9 chars
prefs.putFloat("humidAlert", 70.0);  // 10 chars
prefs.putInt("brightness",   200);   // 10 chars
prefs.putBool("logEnable",  true);   // 9 chars
prefs.putStr("devName",     "E4");   // 7 chars

// Count tool: write key in quotes and count characters
// "brightness" = b-r-i-g-h-t-n-e-s-s = 10 chars ✓
// "temperatureAlert" = 16 chars ✗
```

Key point

Define all NVS keys as constants at the top of your file and count their characters when you define them. Never inline string literals as NVS keys scattered through your code. A constants block makes the 15-character limit easy to audit at a glance.

3.1 Key constant pattern

```
// ── NVS key constants — all ≤15 characters ──────────────────
// Namespace
#define NVS_NS        "fundrum"     // 7 chars

// Keys — count checked
#define NVS_BRIGHT    "brightness"  // 10 chars ✓
#define NVS_TEMP_HI   "tempAlert"   //  9 chars ✓
#define NVS_HUMID_HI  "humidAlert"  // 10 chars ✓
#define NVS_LOG_EN    "logEnable"   //  9 chars ✓
#define NVS_INTERVAL  "logInterval" // 11 chars ✓
#define NVS_DEVNAME   "devName"     //  7 chars ✓
#define NVS_BOOTS     "bootCount"   //  9 chars ✓
#define NVS_TOUCH_CAL "cal"         //  3 chars ✓

// Usage — always use the constant, never the raw string
```

```
prefs.putInt(NVS_BRIGHT, 200);
int b = prefs.getInt(NVS_BRIGHT, 200);
```

4 Namespaces

A namespace is a named container for related NVS keys. It isolates groups of settings from each other, prevents key collisions between unrelated parts of your firmware, and makes it easy to clear a whole group of settings in one operation.

4.1 Namespace rules

- Namespace names follow the same 15-character rule as keys
- The same key name can exist in multiple namespaces without conflict
- prefs.begin(namespace, readOnly) opens a namespace
- prefs.end() closes the currently open namespace
- Only one namespace can be open at a time — always end() before begin() of another

4.2 Multiple namespaces in one project

```
#include <Preferences.h>

Preferences prefs;

// Namespace 1: display settings
void saveDisplaySettings(int brightness, bool dimOnIdle) {
  prefs.begin("display", false);
  prefs.putInt("brightness",  brightness);
  prefs.putBool("dimOnIdle",  dimOnIdle);
  prefs.end();
}

// Namespace 2: alert thresholds
void saveAlertSettings(float tempHi, float humidHi) {
  prefs.begin("alerts", false);
  prefs.putFloat("tempAlert",  tempHi);
  prefs.putFloat("humidAlert", humidHi);
  prefs.end();
}

// Namespace 3: touch calibration (same pattern as T09)
void saveTouchCal(uint16_t* calData) {
  prefs.begin("touch", false);
  prefs.putBytes("cal", calData, 10);  // 5 x uint16_t
  prefs.end();
}

// Clear one namespace without affecting others
void clearAlertSettings() {
  prefs.begin("alerts", false);
  prefs.clear();  // Deletes all keys in "alerts" namespace only
  prefs.end();
  Serial.println("Alert settings cleared.");
}

// Clear a specific key
```

```
void clearBrightness() {
  prefs.begin("display", false);
  prefs.remove("brightness");
  prefs.end();
}
```

Key point

Use one namespace per logical group: display, alerts, network, touch, device. This mirrors the structure of your code and makes factory resets surgical — you can clear just the settings that matter without touching calibration data or network credentials.

5 Building a settings manager

A settings manager is a struct plus load/save/reset functions that encapsulate all NVS access in one place. This is the Fundrum pattern for NVS — clean, auditable, and easy to extend.

Create Settings.h in your project src/ folder:

```
// Settings.h — Fundrum E4 Educator NVS settings manager
#pragma once
#include <Arduino.h>
#include <Preferences.h>

// ── NVS key constants — all ≤15 characters ──────────────────
#define NVS_NS_DISP   "display"
#define NVS_NS_ALERT  "alerts"
#define NVS_NS_DEV    "device"

#define NVS_BRIGHT    "brightness"   // 10 ✓
#define NVS_DIM_IDLE  "dimOnIdle"    //  9 ✓
#define NVS_TEMP_HI   "tempAlert"    //  9 ✓
#define NVS_HUMID_HI  "humidAlert"   // 10 ✓
#define NVS_LOG_EN    "logEnable"    //  9 ✓
#define NVS_INTERVAL  "logInterval"  // 11 ✓
#define NVS_BOOTS     "bootCount"    //  9 ✓

// ── Settings struct with defaults ──────────────────────────
struct Settings {
  // Display
  int   brightness  = 200;
  bool  dimOnIdle   = false;

  // Alerts
  float tempAlert   = 28.0;
  float humidAlert  = 70.0;

  // Logger
  bool  logEnable   = true;
  unsigned long logInterval = 5000;

  // Device
  unsigned long bootCount = 0;
};

class SettingsManager {
public:
  Settings s;

  void load() {
    Preferences p;

    p.begin(NVS_NS_DISP, true);
    s.brightness = p.getInt(NVS_BRIGHT,   s.brightness);
```

```
  s.dimOnIdle  = p.getBool(NVS_DIM_IDLE, s.dimOnIdle);
  p.end();

  p.begin(NVS_NS_ALERT, true);
  s.tempAlert  = p.getFloat(NVS_TEMP_HI,  s.tempAlert);
  s.humidAlert = p.getFloat(NVS_HUMID_HI, s.humidAlert);
  p.end();

  p.begin(NVS_NS_DEV, true);
  s.logEnable   = p.getBool(NVS_LOG_EN,   s.logEnable);
  s.logInterval = p.getULong(NVS_INTERVAL, s.logInterval);
  s.bootCount   = p.getULong(NVS_BOOTS,    0) + 1;
  p.end();

  // Increment boot counter
  saveBootCount();
  Serial.printf("Settings loaded. Boot #%lu\n", s.bootCount);
}

void save() {
  Preferences p;

  p.begin(NVS_NS_DISP, false);
  p.putInt(NVS_BRIGHT,   s.brightness);
  p.putBool(NVS_DIM_IDLE, s.dimOnIdle);
  p.end();

  p.begin(NVS_NS_ALERT, false);
  p.putFloat(NVS_TEMP_HI,  s.tempAlert);
  p.putFloat(NVS_HUMID_HI, s.humidAlert);
  p.end();

  p.begin(NVS_NS_DEV, false);
  p.putBool(NVS_LOG_EN,   s.logEnable);
  p.putULong(NVS_INTERVAL, s.logInterval);
  p.end();

  Serial.println("Settings saved.");
}

void reset() {
  Preferences p;
  p.begin(NVS_NS_DISP,  false); p.clear(); p.end();
  p.begin(NVS_NS_ALERT, false); p.clear(); p.end();
  p.begin(NVS_NS_DEV,   false); p.clear(); p.end();
  s = Settings();  // Reset struct to defaults
  Serial.println("Factory reset complete.");
}

void print() {
  Serial.printf("  Brightness:  %d\n",   s.brightness);
  Serial.printf("  DimOnIdle:   %s\n",   s.dimOnIdle ? "yes" : "no");
  Serial.printf("  TempAlert:   %.1f C\n", s.tempAlert);
```

```
    Serial.printf("  HumidAlert:  %.1f%%\n", s.humidAlert);
    Serial.printf("  LogEnable:   %s\n",   s.logEnable ? "yes" : "no");
    Serial.printf("  LogInterval: %lums\n", s.logInterval);
    Serial.printf("  BootCount:   %lu\n",  s.bootCount);
  }

private:
  void saveBootCount() {
    Preferences p;
    p.begin(NVS_NS_DEV, false);
    p.putULong(NVS_BOOTS, s.bootCount);
    p.end();
  }
};
```

Using SettingsManager in main.cpp:

```
#include "Settings.h"

SettingsManager settings;

void setup() {
  Serial.begin(115200);
  settings.load();
  settings.print();

  // Use settings
  ledcWrite(0, settings.s.brightness);
}

void loop() {
  // Modify a setting
  settings.s.brightness = 150;
  settings.save();  // Persist immediately

  // Factory reset on ENT long press
  // settings.reset();
}
```

6 TFT settings UI

A settings screen is a natural addition to any TFT project. It lets a learner change values at runtime without reflashing, and demonstrates the NVS save/load cycle in a satisfying visible way.

6.1 Settings screen design

The design uses the standard three-zone layout from T08. The content zone shows a list of settings, NEXT cycles through them, ENT short adjusts the selected value, and ENT long saves and returns to the main dashboard.

```
#include <Arduino.h>
#include <TFT_eSPI.h>
#include <Preferences.h>
#include "Button.h"
#include "Settings.h"

TFT_eSPI       tft;
TFT_eSprite    spr = TFT_eSprite(&tft);
Button         nextBtn(BTN_NEXT);
Button         entBtn(BTN_ENT);
SettingsManager settings;

#define COL_BG      0x0000
#define COL_HEADER  0x0319
#define COL_TEXT    0xFFFF
#define COL_LABEL   0xC618
#define COL_SEL     0xFD20   // Selected item highlight
#define COL_OK      0x07E0

enum AppPage { PAGE_MAIN, PAGE_SETTINGS };
AppPage page = PAGE_MAIN;

// Settings items
struct SettingItem {
  const char* label;
  int   minVal, maxVal, step;
};

const SettingItem SETTINGS_ITEMS[] = {
  { "Brightness",  50, 255, 10 },
  { "Temp alert",  20, 40,   1 },
  { "Humid alert", 40, 90,   5 },
  { "Log interval",1,  30,   1 },   // In seconds
};
const int NUM_ITEMS = sizeof(SETTINGS_ITEMS) / sizeof(SETTINGS_ITEMS[0]);
int selectedItem = 0;

int getItemValue(int idx) {
  switch(idx) {
    case 0: return settings.s.brightness;
    case 1: return (int)settings.s.tempAlert;
    case 2: return (int)settings.s.humidAlert;
```

```
    case 3: return (int)(settings.s.logInterval / 1000);
    default: return 0;
  }
}

void setItemValue(int idx, int val) {
  switch(idx) {
    case 0:
      settings.s.brightness = val;
      ledcWrite(0, val);  // Apply immediately
      break;
    case 1: settings.s.tempAlert  = (float)val; break;
    case 2: settings.s.humidAlert = (float)val; break;
    case 3: settings.s.logInterval = (unsigned long)val * 1000; break;
  }
}

void drawSettingsPage() {
  tft.fillScreen(COL_BG);
  tft.fillRect(0, 0, 240, 40, COL_HEADER);
  tft.setTextColor(COL_TEXT, COL_HEADER);
  tft.setTextSize(2);
  tft.setCursor(8, 12);
  tft.print("Settings");

  spr.fillSprite(COL_BG);

  for (int i = 0; i < NUM_ITEMS; i++) {
    int y = i * 52;
    bool sel = (i == selectedItem);

    // Highlight selected row
    if (sel) spr.fillRect(0, y, 240, 48, 0x0A29);

    spr.setTextColor(sel ? COL_SEL : COL_LABEL, sel ? 0x0A29 : COL_BG);
    spr.setTextSize(1);
    spr.setCursor(8, y + 6);
    spr.print(SETTINGS_ITEMS[i].label);

    spr.setTextColor(sel ? COL_TEXT : COL_LABEL, sel ? 0x0A29 : COL_BG);
    spr.setTextSize(2);
    spr.setCursor(8, y + 22);

    int val = getItemValue(i);
    if (i == 0)      spr.printf("%d / 255",  val);
    else if (i == 1) spr.printf("%d C",       val);
    else if (i == 2) spr.printf("%d %%",      val);
    else if (i == 3) spr.printf("%d sec",     val);
  }

  // Footer hints
  spr.setTextColor(COL_LABEL, COL_BG);
  spr.setTextSize(1);
  spr.setCursor(8, 220);
```

```
  spr.print("NEXT=select  ENT=adjust  ENT long=save");

  spr.pushSprite(0, 48);
}

void setup() {
  Serial.begin(115200);
  settings.load();

  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);
  tft.init();
  tft.setRotation(0);   // Landscape — correct for E4 Educator v2.0
  tft.fillScreen(COL_BG);
  ledcSetup(0, 5000, 8);
  ledcAttachPin(TFT_BL, 0);
  ledcWrite(0, settings.s.brightness);

  spr.setColorDepth(16);
  spr.createSprite(320, 165);

  nextBtn.begin();
  entBtn.begin();

  drawSettingsPage();
  Serial.println("Settings UI ready.");
  settings.print();
}

void loop() {
  Button::Event nextEv = nextBtn.read();
  Button::Event entEv  = entBtn.read();

  // NEXT: cycle through settings items
  if (nextEv == Button::SHORT_PRESS) {
    selectedItem = (selectedItem + 1) % NUM_ITEMS;
    drawSettingsPage();
  }

  // ENT short: increment selected value
  if (entEv == Button::SHORT_PRESS) {
    int val = getItemValue(selectedItem);
    val += SETTINGS_ITEMS[selectedItem].step;
    if (val > SETTINGS_ITEMS[selectedItem].maxVal)
      val = SETTINGS_ITEMS[selectedItem].minVal;
    setItemValue(selectedItem, val);
    drawSettingsPage();
  }

  // ENT long: save and confirm
  if (entEv == Button::LONG_PRESS) {
    settings.save();
    // Flash green LED as confirmation
```

```
    digitalWrite(LED_GREEN, HIGH);
    delay(300);
    digitalWrite(LED_GREEN, LOW);
    Serial.println("Settings saved via ENT long.");
    settings.print();
  }
}
```

6.2 What to observe

- NEXT cycles through the four settings items with orange highlight on the selected row
- ENT short increments the selected value with immediate effect — brightness changes the backlight in real time
- ENT long saves all settings to NVS and flashes the GREEN LED as confirmation
- Power cycle the board — all settings are restored exactly as saved
- Serial monitor shows boot count incrementing on each power cycle

Try this

Set brightness to 50 and save. Power cycle the board. The backlight comes up at 50 instead of the 200 default. Change it back to 200 and save. This is NVS persistence in its most satisfying form — the device remembers what you told it.

7 Factory reset

Every device with persistent settings needs a factory reset path — a way to clear all stored values and return to defaults. On the E4, the most natural factory reset trigger is holding both NEXT and ENT simultaneously at startup.

```
// Factory reset detection in setup()
// Hold both buttons at power-on for 3 seconds
void checkFactoryReset() {
  pinMode(BTN_NEXT, INPUT_PULLUP);
  pinMode(BTN_ENT,  INPUT_PULLUP);

  if (digitalRead(BTN_NEXT) == LOW && digitalRead(BTN_ENT) == LOW) {
    Serial.println("Factory reset triggered — hold for 3 seconds...");

    // Show warning on TFT
    tft.fillScreen(TFT_BLACK);
    tft.setTextColor(TFT_RED, TFT_BLACK);
    tft.setTextSize(2);
    tft.setCursor(20, 100);
    tft.print("FACTORY RESET");
    tft.setTextSize(1);
    tft.setCursor(20, 130);
    tft.print("Release to cancel...");

    unsigned long held = millis();
    bool confirmed = false;

    while (digitalRead(BTN_NEXT) == LOW && digitalRead(BTN_ENT) == LOW) {
      if (millis() - held >= 3000) {
        confirmed = true;
        break;
      }
      delay(50);
    }

    if (confirmed) {
      settings.reset();
      tft.setTextColor(TFT_GREEN, TFT_BLACK);
      tft.setCursor(20, 160);
      tft.print("Reset complete. Rebooting...");
      delay(2000);
      ESP.restart();
    } else {
      Serial.println("Factory reset cancelled.");
    }
  }
}

// Call checkFactoryReset() at the start of setup()
// BEFORE loading settings — in case we want to clear them
```

Key point

Always check for factory reset BEFORE loading settings in setup(). If you load settings first and then reset, you end up using the old values during the current boot cycle. The correct order is: detect reset trigger → clear NVS if confirmed → then load (which will return defaults since NVS is now empty).

8 T11 summary

Concept	Key takeaway
NVS vs SD	NVS: small settings, always available, no card needed. SD: large data, logging, images. Use both together.
15-char key rule	Keys silently truncate at 15 characters. Collisions cause silent overwrites. Define all keys as constants and count characters.
Default values	Always provide a default in every get() call. First-boot behaviour is predictable and safe.
Read-only mode	prefs.begin(ns, true) for reading. prefs.begin(ns, false) for writing. Use read-only wherever possible.
Namespaces	One namespace per logical group. clear() wipes one namespace without touching others. ≤15 chars same rule as keys.
Settings.h pattern	Struct with defaults + load/save/reset/print functions. Single source of truth for all NVS access in the project.
Factory reset	Check before loading settings. Hold both buttons at boot. Clear NVS namespaces. Restart. Show TFT feedback.
Boot counter	Increment in load(), save immediately after. Useful diagnostic — shows total power cycles since manufacture.

What's next

- T12 — WiFi and MQTT: with local storage (NVS + SD) established, the natural next step is connecting the E4 to a network. WiFi credentials stored in NVS, sensor data published to an MQTT broker, and the Node-RED dashboard receiving live readings from the E4.